TASK 1: Executed The Script shown in the video.

Terraform Task: Creating an AWS EC2 Instance Using Terraform Modules

1. Objective

To create an AWS EC2 instance using Terraform by implementing a reusable module and passing an AMI ID from the root configuration to the child module through an input variable.

2. Purpose

The purpose of this task is to understand how Terraform modules work and how they help organize and reuse infrastructure code.

This task helps to learn how to:

- Create a reusable Terraform module.
- Define input variables in Terraform.
- Pass an AMI ID from a root module to a child module.
- Configure an AWS EC2 instance.
- Execute Terraform commands to create and manage AWS resources.

3. Project Structure

The project contains a root configuration and a child module.

text

```
terraform_Basics/
|
 ├── aws_instance/
|    ├── main.tf
|   └── variable.tf
|
└── Dev/
    ├── main.tf
    ├── .terraform/
    ├── .terraform.lock.hcl
   └── terraform.tfstate
```

File descriptions:

- aws_instance/main.tf: Defines the AWS EC2 instance resource.
- aws_instance/variable.tf: Declares the AMI input variable.
- Dev/main.tf: Calls the child module and passes the AMI ID.
- .terraform.lock.hcl: Records the selected provider versions.
- terraform.tfstate: Tracks the resources managed by the root configuration.

4. Implementation Steps

Step 1: Create the child module

Create the aws_instance folder and add the following files.

Step 2: Configure the EC2 resource

In aws_instance/main.tf, write the following code:

```
resource "aws_instance" "webserver" {
  ami           = var.ami
  instance_type = "t3.micro"

  tags = {
    Name = "Module Instance"
  }
}
```

Explanation:

- resource "aws_instance" "webserver" defines an AWS EC2 instance.
- ami = var.ami uses the AMI ID supplied through an input variable.
- instance_type = "t3.micro" specifies the EC2 instance type.
- tags assigns the name Module Instance to the EC2 instance.

Step 3: Declare the input variable

In aws_instance/variable.tf, write:

```
variable "ami" {


}
```

Explanation:

- variable "ami" declares an input variable named ami.
- type = string specifies that the variable accepts a single string value.

This allows the child module to receive the AMI ID from the root configuration.

Step 4: Configure the root module

In Dev/main.tf, write:

```
module "dev_server" {
  source = "../aws_instance"
  ami    = "ami-041607313c6a2a618"
}
```

Explanation:

- module "dev_server" defines a module call.
- source = "../aws_instance" specifies the relative path to the child module.
- ami = "ami-041607313c6a2a618" passes the AMI ID to the child module.

The root module passes the value to the child module, where var.ami uses it to configure the EC2 instance.

The AMI ID must be valid and available in the AWS region configured for the provider.

Step 5: Initialize Terraform

Open Git Bash and navigate to the Dev directory: cd ~/OneDrive/Desktop/terraform_Basics/Dev

Initialize Terraform: terraform init

This command initializes the working directory, downloads the required providers, and prepares the module configuration.

Step 6: Format the configuration: terraform fmt

This command formats the Terraform files according to standard formatting conventions.

Step 7: Validate the configuration: terraform validate

This command checks the Terraform configuration for syntax and structural errors.

Step 8: Review the execution plan: terraform plan

Step 9: Apply the configuration: terraform apply

Review the execution plan and enter: yes

Step 10: Verify the resource: terraform state list

The expected resource address is: module.dev_server.aws_instance.webserver

You can also open the AWS Management Console, navigate to EC2, and verify that the instance exists in the configured region.

5. Important Terraform Concepts

Terraform module

A module is a collection of Terraform configuration files used to organize and reuse infrastructure code. In this task, the aws_instance directory acts as a child module.

Input variable

An input variable allows a module to receive configuration values from another module. Here, the ami variable receives the AMI ID from the root configuration.

Module source

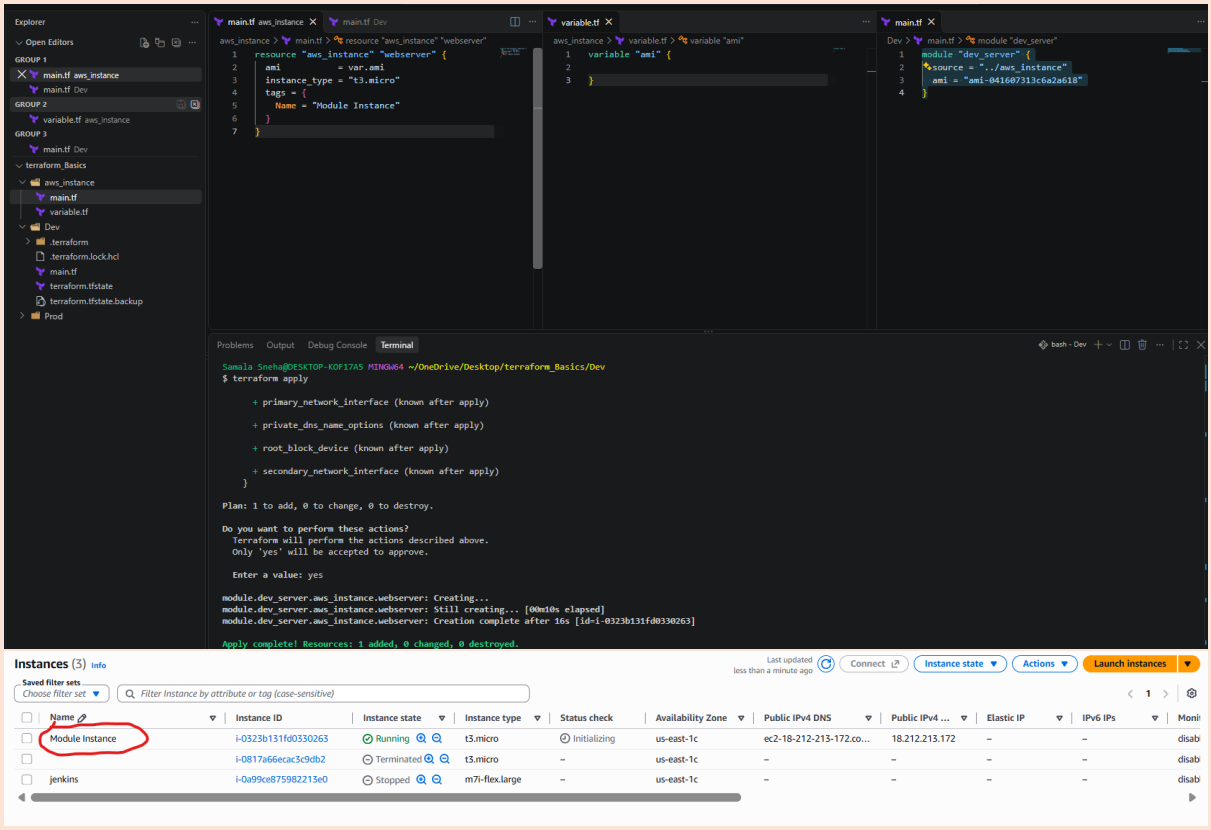
The source argument identifies where Terraform can find the child module. The relative path ../aws_instance points to the module directory.

Terraform workflow

The main commands used in this task are:

1. terraform init — initializes Terraform.
2. terraform fmt — formats the configuration.
3. terraform validate — validates the configuration.
4. terraform plan — previews infrastructure changes.
5. terraform apply — applies the configuration.
6. terraform state list — lists resources tracked in the current state.

6. Implementation



7. Conclusion

In this task, an AWS EC2 instance was successfully created using Terraform modules. The EC2 resource was defined in a child module, and the AMI ID was passed from the root configuration using an input variable.

The terraform init, terraform validate, terraform plan, and terraform apply commands were used to initialize, validate, preview, and create the infrastructure.

The final output confirmed that one resource was added successfully. This task demonstrates how Terraform modules improve code organization, reusability, and infrastructure management.

Terraform Workspace Task: Managing AWS EC2 Instances Using Terraform Workspaces

1. Objective

To understand and implement Terraform workspaces for managing separate infrastructure environments and create AWS EC2 instances using Terraform variables and the for_each meta-argument.

2. Purpose

The purpose of this task is to learn how to:

- Create and manage Terraform workspaces.
- Understand how workspaces maintain separate Terraform state files.
- Define a map variable containing multiple AMI IDs.
- Use for_each to create multiple EC2 instances.
- Execute Terraform commands to initialize, plan, apply, and manage infrastructure.

3. Implementation Steps

Step 1: Configure AWS credentials

Run the following command: aws configure
Verify the AWS identity: aws sts get-caller-identity

Step 2: Define the map variable

Create variable.tf:

```
variable "ami" {
  type = map(string)

  default = {
    projectA = "ami-041607313c6a2a618"
    projectB = "ami-041607313c6a2a618"
  }
}
```

Explanation:

- ami is the variable name.
- map(string) stores multiple string values associated with keys.
- projectA and projectB identify the entries.
- Each entry contains an AMI ID.

Step 3: Define the EC2 resource

Create main.tf:

```
resource "aws_instance" "webserver" {
 for_each = var.ami

 ami          = each.value
 instance_type = "t3.micro"

 tags = {
  Name = "WorkspaceInstance-${each.key}"
 }
}
```

Explanation:

- for_each = var.ami creates an instance for every entry in the map.
- each.key identifies the project.
- each.value supplies the AMI ID.
- instance_type specifies the EC2 instance type.
- The Name tag distinguishes the instances.

Step 4: Initialize Terraform terraform init

Step 5: Create a new workspace

terraform workspace new projectA

This creates a workspace named projectA and automatically switches to it.

A workspace maintains its own Terraform state, separate from other workspaces.

Step 6: Verify the current workspace: terraform workspace show

To list all available workspaces: terraform workspace list

Step 7: Format and validate the configuration

Format the configuration: terraform fmt

Validate the configuration: terraform validate

These commands help ensure the Terraform code is properly formatted and structurally valid.

Step 8: Review the execution plan: terraform plan

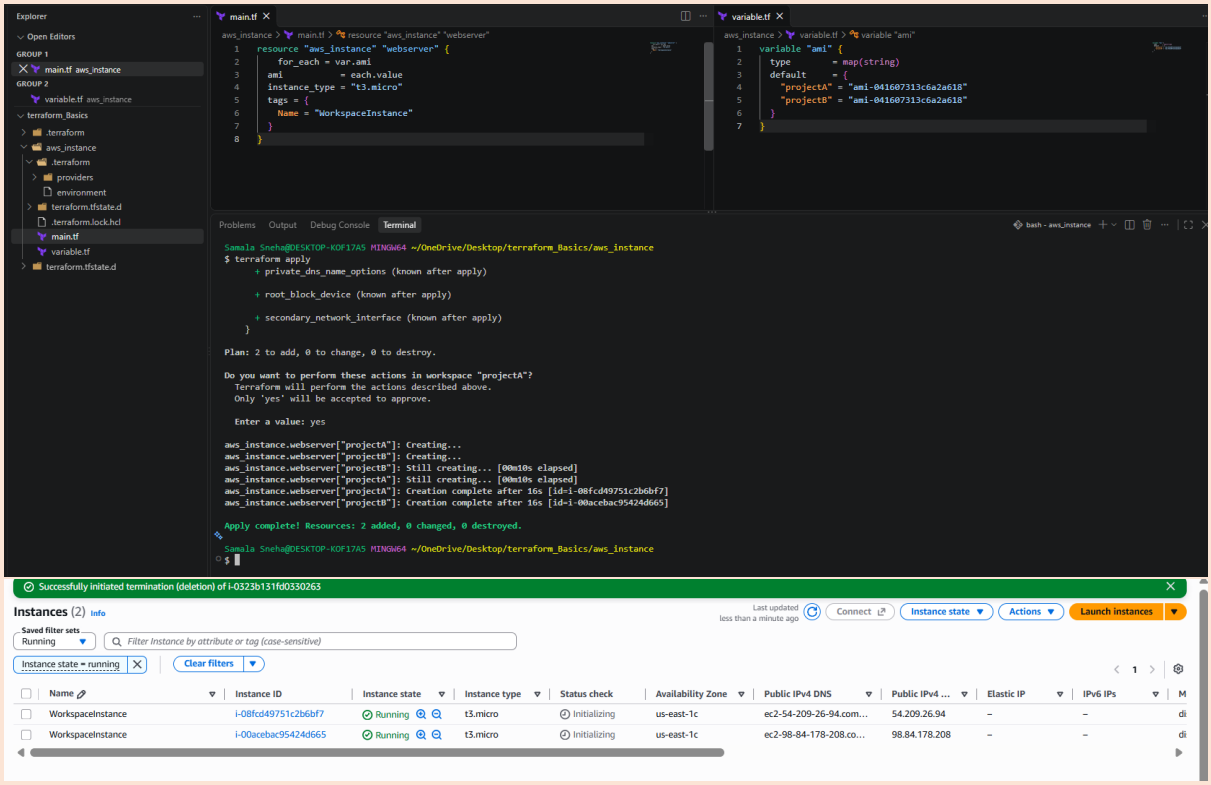
Step 9: Apply the configuration: terraform apply

Step 10: Verify the resources: terraform state list

aws_instance.webserver["projectA"]

aws_instance.webserver["projectB"]

4. Implementation



5. Conclusion

In this task, Terraform workspaces were used to manage infrastructure state independently, while a map variable and the for_each meta-argument were used to define multiple AWS EC2 instances without modules.

The task demonstrates the use of Terraform workspaces, map variables, resource iteration, and the standard Terraform workflow. Workspaces help organize separate environments, while for_each reduces repetitive code when creating multiple resources.

Task 2:Terraform Project: Provision EC2, S3, and VPC Using Modules

1. Objective

To provision AWS infrastructure using Terraform modules by creating a Virtual Private Cloud (VPC), an EC2 instance, and an S3 bucket. The project demonstrates how to organize Terraform configurations into reusable modules and manage AWS resources efficiently.

2. Purpose

- Learn how to create and use reusable Terraform modules.
- Create a VPC with a subnet, internet gateway, and routing configuration.
- Launch an EC2 instance inside the VPC.
- Create an S3 bucket for object storage.
- Pass configuration values between the root module and child modules.
- Use Terraform commands to initialize, validate, plan, apply, and manage infrastructure.

3. Project folder structure

Create the following structure in your terraform_Basics folder:

text

```
terraform_Basics/
 ├── main.tf
 ├── variables.tf
 ├── outputs.tf
 ├── providers.tf
|
└── modules/
    ├── vpc/
    |    ├── main.tf
    |    ├── variables.tf
    |    └── outputs.tf
    |
    ├── ec2/
    |    ├── main.tf
    |    ├── variables.tf
    |    └── outputs.tf
    |
    └── s3/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

Each child module manages one type of AWS resource, while the root module connects the components.

4. Implementation

Git Bash script Copy and paste this entire script into Git Bash:

```
mkdir -p EC2_S3_VPC/modules/{vpc,ec2,s3}

touch EC2_S3_VPC/main.tf
touch EC2_S3_VPC/variables.tf
touch EC2_S3_VPC/outputs.tf

touch EC2_S3_VPC/modules/vpc/{main.tf,variables.tf,outputs.tf}
touch EC2_S3_VPC/modules/ec2/{main.tf,variables.tf,outputs.tf}
touch EC2_S3_VPC/modules/s3/{main.tf,variables.tf,outputs.tf}

cd EC2_S3_VPC


echo "Project folder structure created successfully!"
```

Verify the structure: find . -type f

Step 1: Configure the AWS provider

Create **providers.tf** in the root directory.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

This configures Terraform to use the AWS provider in the selected region.

Step 2: Create the VPC module

Create **modules/vpc/main.tf**.

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "Terraform-VPC"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "Terraform-Public-Subnet"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "Terraform-IGW"
```

```
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "Terraform-Public-RT"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

Create **modules/vpc/variables.tf**.

```
variable "vpc_cidr" {
  type    = string
  default = "10.0.0.0/16"
}

variable "subnet_cidr" {
  type    = string
  default = "10.0.1.0/24"
}

variable "availability_zone" {
  type    = string
  default = "ap-south-1a"
}
```

Create **modules/vpc/outputs.tf**.

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "subnet_id" {
  value = aws_subnet.public.id
}
```

Purpose: This module creates a VPC, public subnet, internet gateway, and route table. The outputs provide the VPC and subnet IDs to the EC2 module.

Step 3: Create the EC2 module

Create **modules/ec2/main.tf**.

```
data "aws_ssm_parameter" "ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

resource "aws_security_group" "web" {
  name        = "terraform-ec2-sg"
  description = "Security group for Terraform EC2"
  vpc_id      = var.vpc_id

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "webserver" {
  ami                    = data.aws_ssm_parameter.ami.value
  instance_type          = var.instance_type
  subnet_id              = var.subnet_id
  vpc_security_group_ids = [aws_security_group.web.id]
  associate_public_ip_address = true

  tags = {
    Name = "Terraform-EC2"
  }
}
```

Create **modules/ec2/variables.tf**.

```
variable "vpc_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}
```

Create modules/ec2/outputs.tf.

```
hcl
output "instance_id" {
```

```
  value = aws_instance.webserver.id
}

output "public_ip" {
  value = aws_instance.webserver.public_ip
}
```

Purpose: This module launches an EC2 instance in the subnet created by the VPC module. It retrieves a current Amazon Linux 2023 AMI using AWS Systems Manager.

The security group allows outbound traffic but does not allow inbound SSH or HTTP connections. You can add restricted inbound rules later if required.

Step 4: Create the S3 module

Create **modules/s3/main.tf**.

```
resource "aws_s3_bucket" "storage" {
  bucket = var.bucket_name

  tags = {
    Name = "Terraform-Storage"
  }
}

resource "aws_s3_bucket_public_access_block" "storage" {
  bucket = aws_s3_bucket.storage.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "storage" {
  bucket = aws_s3_bucket.storage.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

Create **modules/s3/variables.tf**.

```
variable "bucket_name" {
  type = string
}
```

Create **modules/s3/outputs.tf**.

```
output "bucket_name" {
  value = aws_s3_bucket.storage.bucket
}

output "bucket_arn" {
  value = aws_s3_bucket.storage.arn
}
```

Purpose: This module creates an S3 bucket with public access blocked and default server-side encryption enabled.

Choose a globally unique bucket name, such as terraform-storage-youruniqueid-2026.

Step 5: Configure root variables

Create **variables.tf** in the root directory.

```
variable "aws_region" {
  type    = string
  default = "ap-south-1"
}

variable "bucket_name" {
  type = string
}
```

The bucket name must be globally unique across AWS. The default VPC module availability zone is ap-south-1a, which corresponds to the Mumbai region.

Step 6: Call all three modules

Create **main.tf** in the root directory.

```
module "vpc" {
  source = "./modules/vpc"
}

module "ec2" {
  source = "./modules/ec2"

  vpc_id       = module.vpc.vpc_id
  subnet_id    = module.vpc.subnet_id
  instance_type = "t3.micro"
}
```

```
module "s3" {
  source = "./modules/s3"

  bucket_name = var.bucket_name
}
```

This is the main configuration that connects all three modules.

- The VPC module creates the network.
- The EC2 module receives the VPC and subnet IDs from the VPC module.
- The S3 module creates a storage bucket using the supplied bucket name.

Step 7: Define root outputs

Create **outputs.tf** in the root directory.

```
output "vpc_id" {
  value = module.vpc.vpc_id
}

output "ec2_instance_id" {
  value = module.ec2.instance_id
}

output "ec2_public_ip" {
  value = module.ec2.public_ip
}

output "s3_bucket_name" {
  value = module.s3.bucket_name
}
```

These outputs display the IDs, public IP, and bucket name after deployment.

5. Execute Terraform commands

Initialize Terraform: terraform init

Format the files: terraform fmt -recursive

Validate the configuration: terraform validate

Provide a unique S3 bucket name

Create a file named **terraform.tfvars** in the root directory:
bucket_name = "terraform-storage-youruniqueid-2026"

Replace the example with a globally unique name before proceeding.

Review the execution plan: terraform plan

Create the infrastructure: terraform apply

Review the proposed changes and enter yes when prompted if you approve them.

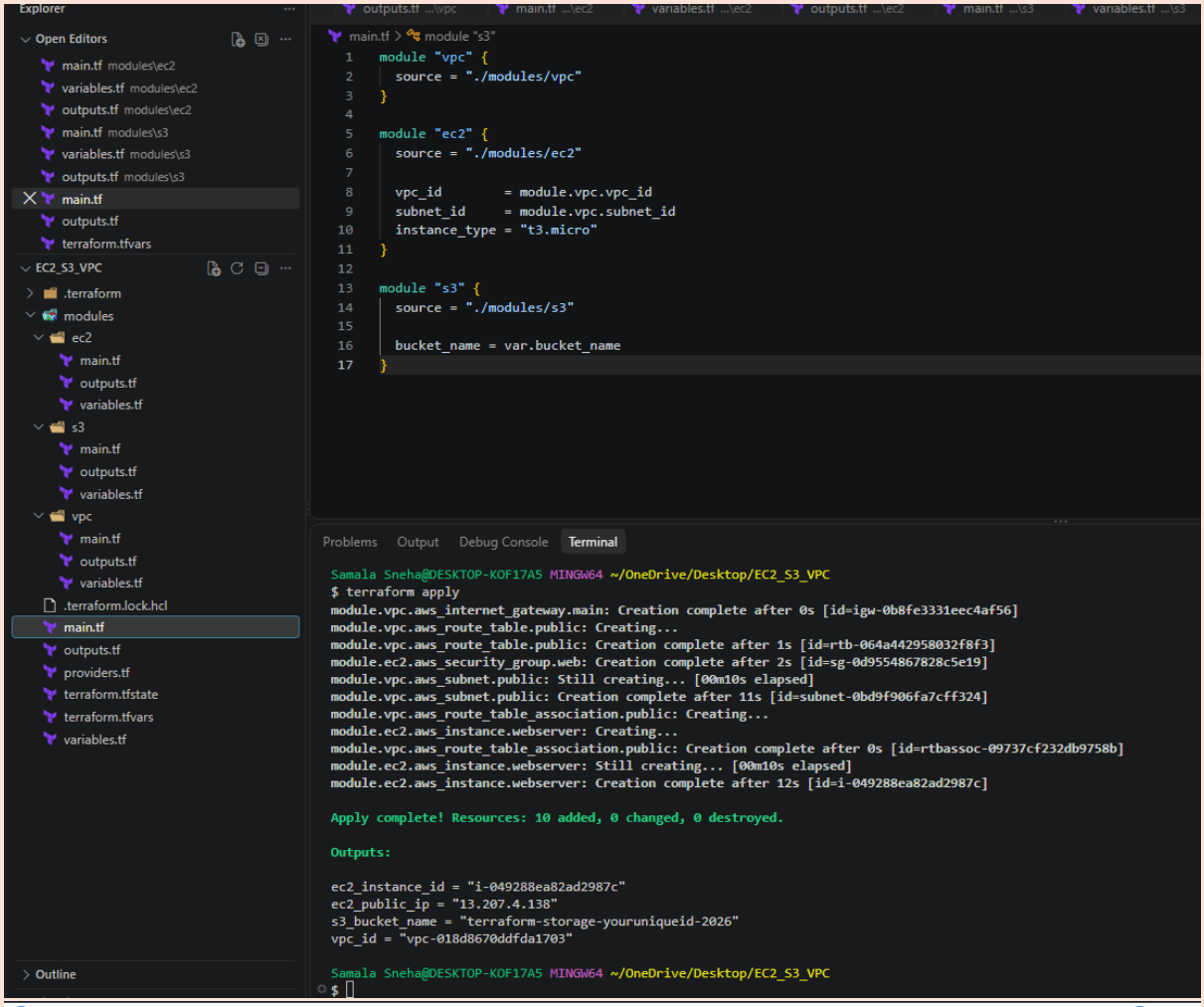
Verify the resources

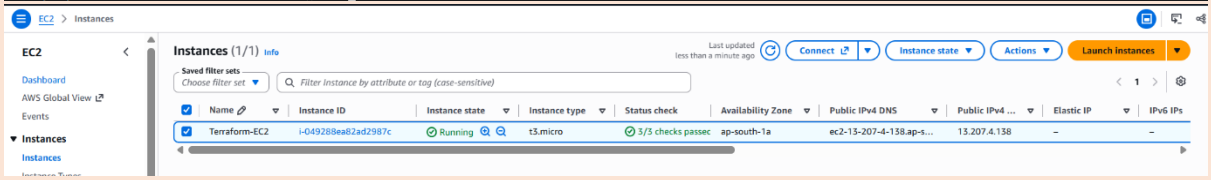
→ terraform output
→ terraform state list
Expected resource addresses will include resources under:

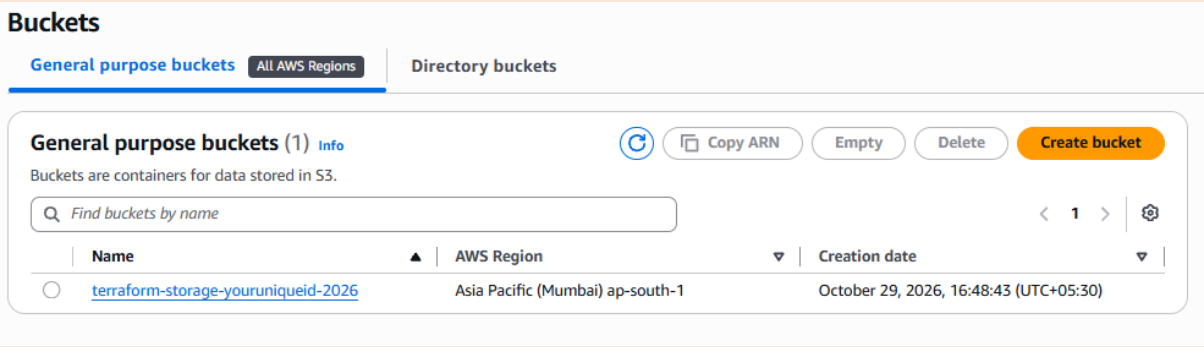
- module.vpc
- module.ec2
- module.s3

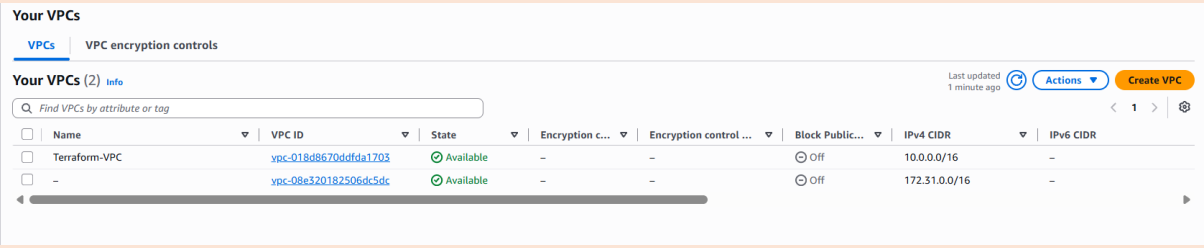
The exact resource count depends on the configuration and existing state.

6. Implementation









7. Conclusion

In this project, Terraform modules are used to provision three major AWS services: EC2, S3, and VPC. The VPC module creates the networking infrastructure, the EC2 module launches a virtual server within that network, and the S3 module provides secure object storage.

This project demonstrates modular infrastructure design, input and output variables, dependencies between modules, and the Terraform workflow for managing AWS resources.

3). Provision **EC2 for 3 different environments** (Dev, Staging, and Prod) using **Terraform workspaces**.

Terraform Task: Provision EC2 Instances for Dev, Staging, and Prod Using Terraform Workspaces

1. Objective

To provision separate AWS EC2 instances for three environments—Development (Dev), Staging, and Production (Prod)—using Terraform workspaces.

2. Purpose

The purpose of this task is to understand how Terraform workspaces help manage separate infrastructure environments using the same Terraform configuration.

This task aims to:

- Create and manage Terraform workspaces.
- Provision separate EC2 instances for Dev, Staging, and Prod.
- Use terraform.workspace to identify the active environment.
- Apply environment-specific configuration and resource tags.
- Understand how Terraform maintains separate state for each workspace.
- Use Terraform commands to initialize, validate, plan, apply, and verify infrastructure.

3. Project Structure

The project uses a single Terraform configuration without modules.

text

```
EC2_Workspaces/
 ├── main.tf
 ├── variables.tf
 └── outputs.tf
```

File descriptions:

- main.tf: Configures the AWS provider, retrieves the AMI, identifies the active workspace, and defines the EC2 instance.
- variables.tf: Defines the AWS region variable.
- outputs.tf: Displays the active workspace, EC2 instance ID, and public IP address.

4. Implementation Steps

Step 1: Create the project directory

Create the project folder and Terraform files using Git Bash:

```
mkdir -p EC2_Workspaces
cd EC2_Workspaces

touch main.tf
touch variables.tf
touch outputs.tf
```

Step 2: Configure the AWS provider and EC2 resource

Write the following code in **main.tf**:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_ssm_parameter" "ami" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}

locals {
  environment_config = {
```

```
    default = {
      instance_type = "t3.micro"
    }
    Dev = {
      instance_type = "t3.micro"
    }
    Staging = {
      instance_type = "t3.micro"
    }
    Prod = {
      instance_type = "t3.micro"
    }
  }

  environment = terraform.workspace
}

resource "aws_instance" "webserver" {
  ami           = data.aws_ssm_parameter.ami.value
  instance_type = local.environment_config[local.environment].instance_type

  tags = {
    Name        = "${local.environment}-Server"
    Environment = local.environment
  }
}
```

Explanation:

- terraform specifies the required AWS provider.
- provider "aws" configures the AWS region.
- aws_ssm_parameter.ami retrieves the latest Amazon Linux 2023 AMI ID through AWS Systems Manager.
- environment_config defines the EC2 instance type for each workspace.
- terraform.workspace returns the currently selected workspace name.
- aws_instance.webserver creates an EC2 instance.
- The Name and Environment tags identify the environment associated with the instance.

All three environments use the t3.micro instance type in this exercise.

Step 3: Define the AWS region variable

Write the following code in **variables.tf**:

```
variable "aws_region" {
  description = "AWS region for the EC2 instances"
  type        = string
  default     = "ap-south-1"
}
```

The region is set to Mumbai (ap-south-1).

Step 4: Define Terraform outputs

Write the following code in **outputs.tf**:

```
output "workspace" {
  value = terraform.workspace
}

output "instance_id" {
  value = aws_instance.webserver.id
}

output "public_ip" {
  value = aws_instance.webserver.public_ip
}
```

These outputs display the active workspace, the EC2 instance ID, and the public IP address after deployment.

Step 5: Initialize Terraform

Navigate to the project directory and run: terraform init

This initializes the working directory and downloads the required providers.

Step 6: Format and validate the configuration :

→ terraform fmt

→ terraform validate

- terraform fmt formats the Terraform files.
- terraform validate checks the configuration for syntax and structural errors.

Step 7: Create the Dev workspace

→ terraform workspace new Dev
→ terraform workspace show
→ terraform plan

The terraform workspace new Dev command creates the workspace and switches to it.

The execution plan should propose one EC2 instance with the following tags:

text

```
Name        = "Dev-Server"
Environment = "Dev"
```

Review the plan and create the instance: terraform apply

Enter yes when prompted if you approve the changes.

Step 8: Create the Staging workspace

→ terraform workspace new Staging
→ terraform workspace show
→ terraform plan
The plan should propose a separate EC2 instance with these tags:

text

```
Name        = "Staging-Server"
Environment = "Staging"
```

Create the instance: terraform apply

Review the plan and enter yes when prompted.

Step 9: Create the Prod workspace

→ terraform workspace new Prod
→ terraform workspace show
→ terraform plan
The plan should propose a separate EC2 instance with these tags:

text

```
Name        = "Prod-Server"
Environment = "Prod"
```

Create the instance: terraform apply

Review the plan and enter yes when prompted.

Step 10: Verify the workspaces

List the available workspaces: terraform workspace list
Expected output after creating all three workspaces:

text

```
  default
  Dev
  Staging
* Prod
```

The asterisk indicates the currently selected workspace.

Switch to each workspace and verify its resources.

For Dev:

→ terraform workspace select Dev
→ terraform state list
→ terraform output

For Staging:

→ terraform workspace select Staging
→ terraform state list
→ terraform output

For Prod:

→ terraform workspace select Prod
→ terraform state list
→ terraform output

Each workspace should maintain its own EC2 instance in its own Terraform state.

5. Expected Output

Environment	Workspace	EC2 Name tag	Instance type
Development	Dev	Dev-Server	t3.micro
Staging	Staging	Staging-Server	t3.micro
Production	Prod	Prod-Server	t3.micro

If all three workspace deployments succeed, three EC2 instances should exist in the configured AWS account and region.

The instance IDs and public IP addresses will be different for each environment.

6. Important Terraform Concepts

Terraform workspaces

Workspaces allow the same Terraform configuration to maintain multiple independent state instances.

terraform.workspace

This expression returns the name of the currently selected workspace. It allows the configuration to assign environment-specific values and resource tags.

Local values

The locals block defines reusable values within the configuration. Here, it stores the instance type associated with each workspace.

Separate Terraform state

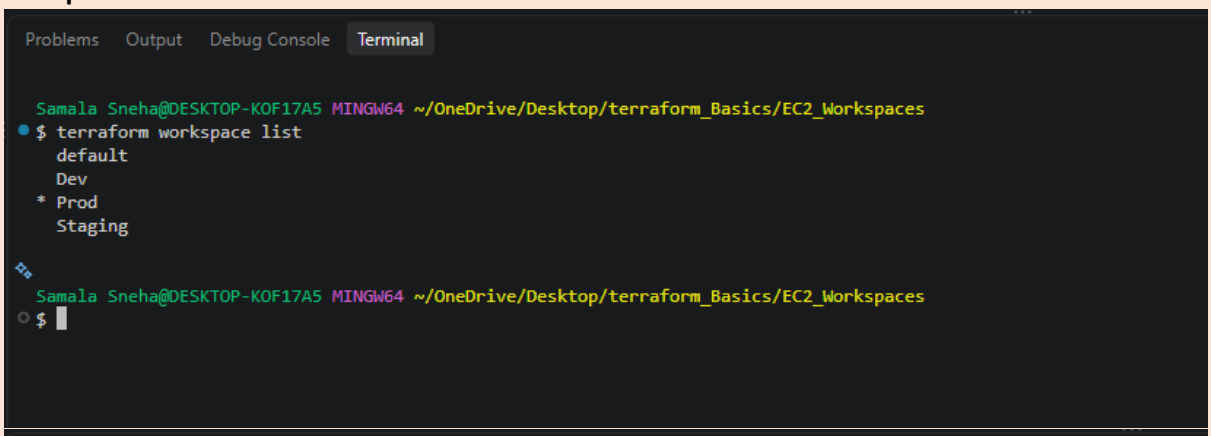
Each workspace maintains its own state. For example, the Dev workspace tracks its Dev EC2 instance, while the Staging and Prod workspaces track their respective instances.

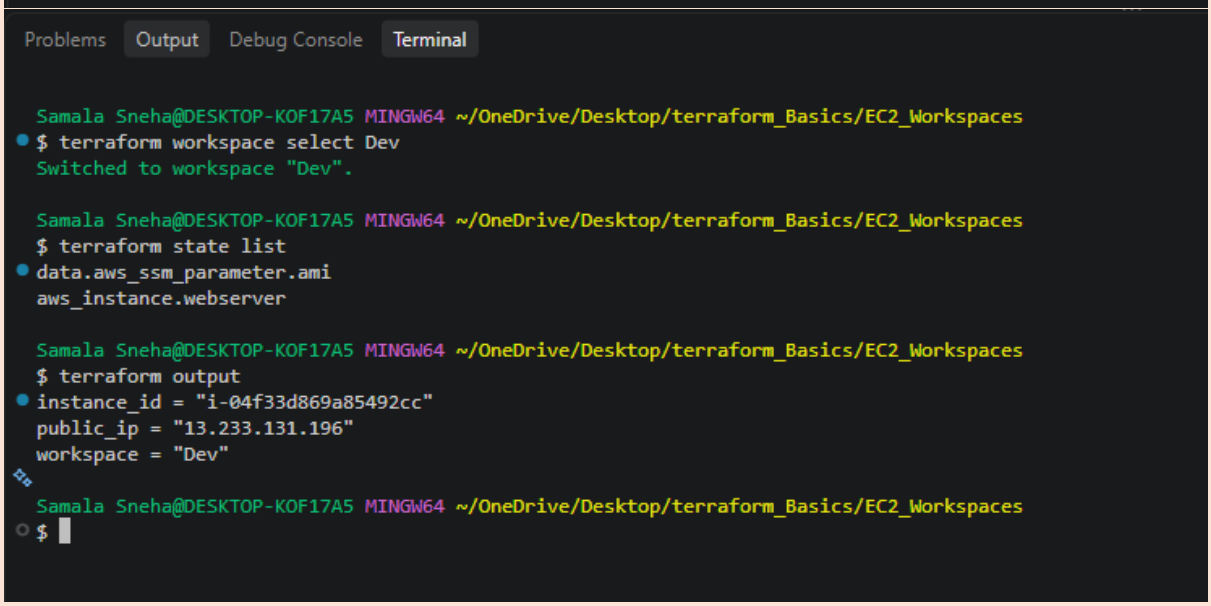
Terraform workflow

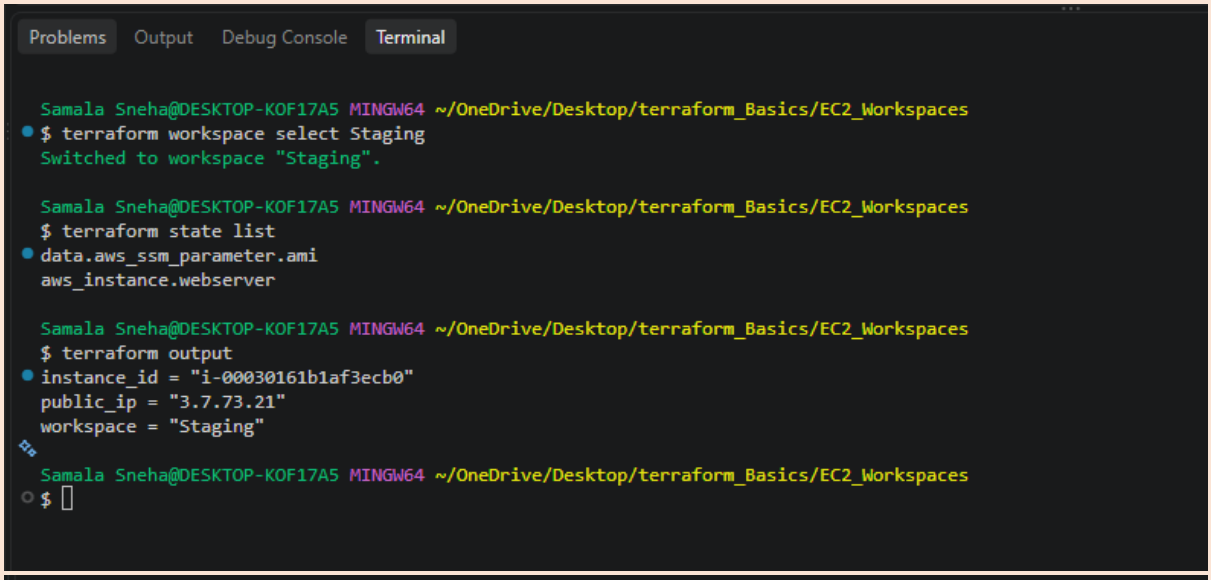
The main commands used are:

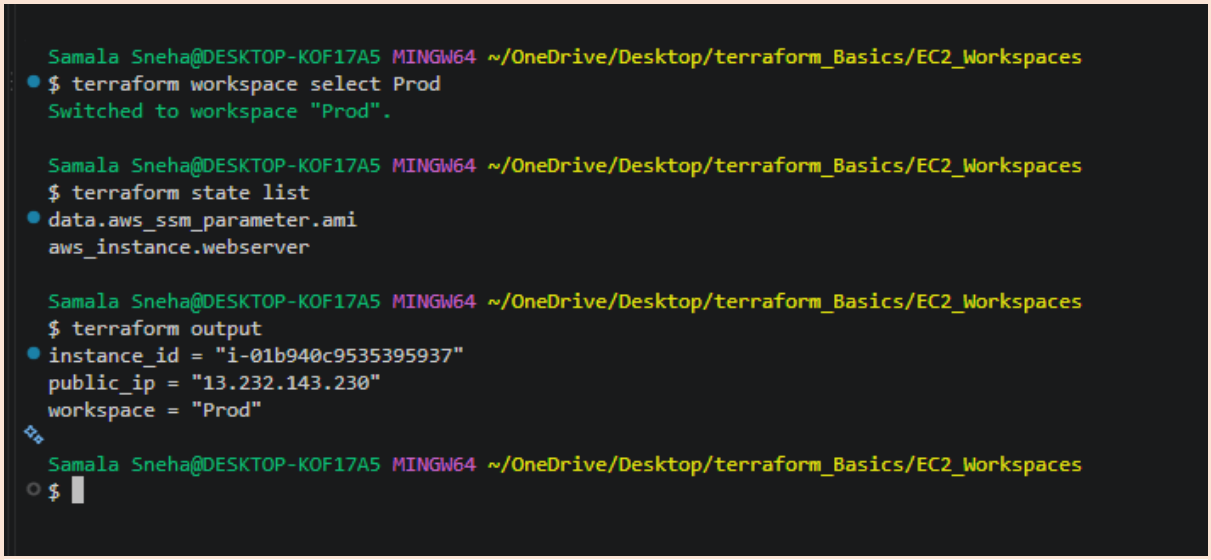
1. terraform init — initializes Terraform.
2. terraform fmt — formats configuration files.
3. terraform validate — validates the configuration.
4. terraform workspace new — creates and selects a workspace.
5. terraform plan — previews infrastructure changes.
6. terraform apply — creates or updates infrastructure.
7. terraform workspace select — switches between workspaces.
8. terraform state list — lists resources tracked by the active workspace.

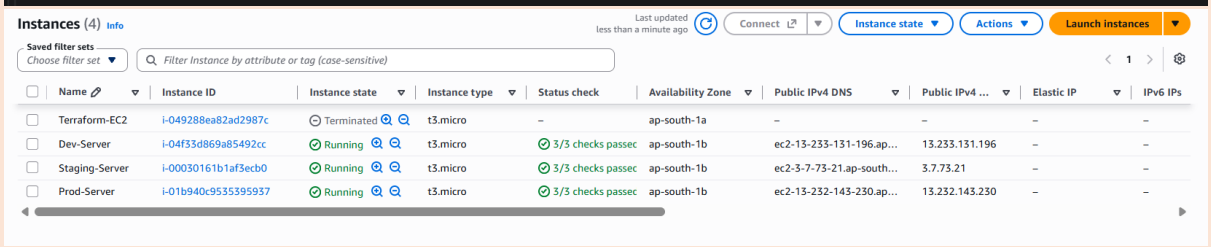
7. Implementation











8. Conclusion

In this task, Terraform workspaces were used to provision separate EC2 instances for Development, Staging, and Production environments using a single Terraform configuration.

The terraform.workspace expression identifies the active environment, while local values and resource tags customize the instance configuration. Separate workspaces maintain separate Terraform states, allowing each environment to be managed independently.

This task demonstrates how Terraform workspaces can simplify environment management and reduce duplicated configuration when deploying infrastructure across multiple environments.

Yes! You can destroy both Terraform projects to avoid ongoing AWS charges. Since you created resources in two separate projects, you must destroy each project separately.

1. Destroy the EC2 workspaces project

Open Git Bash and navigate to your workspace project:
cd ~/OneDrive/Desktop/terraform_Basics/EC2_Workspaces

Check the available workspaces: terraform workspace list

Destroy the resources in each workspace individually.

Dev:
terraform workspace select Dev
terraform destroy

Staging:
terraform workspace select Staging
terraform destroy

Prod:
terraform workspace select Prod
terraform destroy

Type yes.

Each workspace has its own state, so destroying one workspace does not destroy the resources tracked by the others.

2. Destroy the previous EC2, S3, and VPC project

Navigate to your previous project: cd ~/OneDrive/Desktop/EC2_S3_VPC

Run: terraform destroy

Review the plan carefully. It should include the resources created by the VPC, EC2, and S3 modules.

3. Verify the cleanup

For the workspace project, select each workspace and run:

terraform workspace select Dev
terraform state list
Repeat for Staging and Prod. An empty result means no resources are tracked in that workspace's state.

For the previous project: cd ~/OneDrive/Desktop/EC2_S3_VPC
terraform state list

Also check the AWS Console in the Mumbai region (ap-south-1) to confirm that the EC2 instances and VPC have been removed, and check S3 to confirm whether the bucket was deleted.

Remember: terraform destroy removes infrastructure, not your .tf configuration files or workspace definitions. You can keep the code for future practice.